PRODUCT REQUIREMENTS

implemented pure Python

A two-layer MNIST network

How a small net turns 784 pixels into a digit, written so the code can follow the math.

Source	`prd.md`, typeset here as `prd.tex`
Code	`two_layer_mnist.py`
Stories	`user-stories.md`
Reader	Vincent, learning the mechanism by building it

Problem

Vincent wants a very simple neural network in Python so he can learn how it works: a from-scratch two-layer MNIST network. In run mode he wants a clear view of both layers — the 32 hidden neurons and the 10 output scores — without the training-loss log.

The priority is that the loop is obvious. Accuracy is not a goal.

Goals

- A small, self-explanatory network in Python that makes the mechanics visible.
- The definition from Vincent's notes: 784 inputs, 32 hidden neurons, 10 outputs.
- Train on a small MNIST subset and print a few digit guesses.
- Run mode: once weights exist, show each test digit with LAYER[1] and LAYER[2].
- Spec-driven: this document is the source of truth. It runs on Vincent's machine.

Non-goals

- Any language other than Python.
- Any ML library (PyTorch, TensorFlow, Keras, scikit-learn, JAX, and similar).
- NumPy or other numeric libraries. Pure Python only.
- The full 60,000 / 10,000 MNIST training set.
- Production OCR, GPUs, or a deep-learning framework.
- An accuracy target, a hyperparameter search, or tuning for a score.

Theory

This is the teaching core. `two_layer_mnist.py` should read as a walk through of what follows.

What the network is

A neural network here is not a black box. It is two tables of numbers (the weights) plus two small functions (the activations).

We show it a picture of a handwritten digit. It answers with 10 scores, one per digit $0, \dots, 9$. The largest score is its guess.

It does not see a picture the way we do. The picture becomes 784 numbers: each pixel of the 28×28 grid is one number between 0 (white) and 1 (black), laid out in a single row. One image is a row of length 784. A training batch of 150 images is the matrix X of shape 150×784.

What we want to recognize

The goal is not a full page and not a printed font. Given one small handwritten numeral, name the digit.

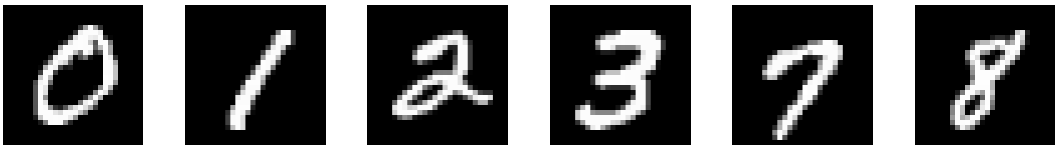


Figure 1: Real MNIST digits from this project. The caption is the true label: what the network should guess after it looks at the picture.

Handwriting varies — a sloppy 7 against a crossed 7, an 8 that looks like a 0. That is why training uses many examples. After training we print a few guesses next to labels like these. We do not require a score. We want the loop to be obvious.

A neuron

One neuron does two things.

1. **Weighted sum.** Multiply each incoming number by a weight and add the products. That sum is z.
2. **Activation.** Pass z through a small function $\sigma(z)$ so the output stays in a useful range.

The weights are the knobs we learn. They start random. After training they should push the digit scores toward the labels.

This project has no bias terms. There is no extra $+b$ on a neuron, so the theory stays aligned with the two weight matrices only.

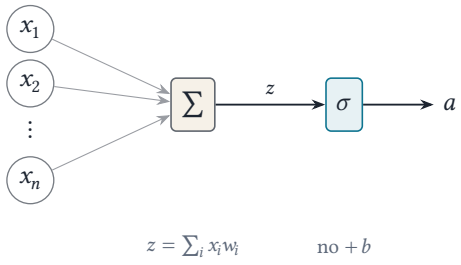


Figure 2: One neuron. The weighted sum z is sent through σ. Hidden neurons use ReLU; output neurons use the sigmoid. Nothing is added after the sum.

Two layers

- **Layer 1, hidden.** 32 neurons. Each one looks at all 784 pixels. Together they are a 32-number internal sketch of the image: not a digit yet, just features the second layer can use.
- **Layer 2, output.** 10 neurons. Each one looks at those 32 numbers and votes for one digit.

There is one hidden layer and one output layer — two layers in total. "Both layers" in run mode means LAYER^1 and LAYER^2, not two hidden layers.

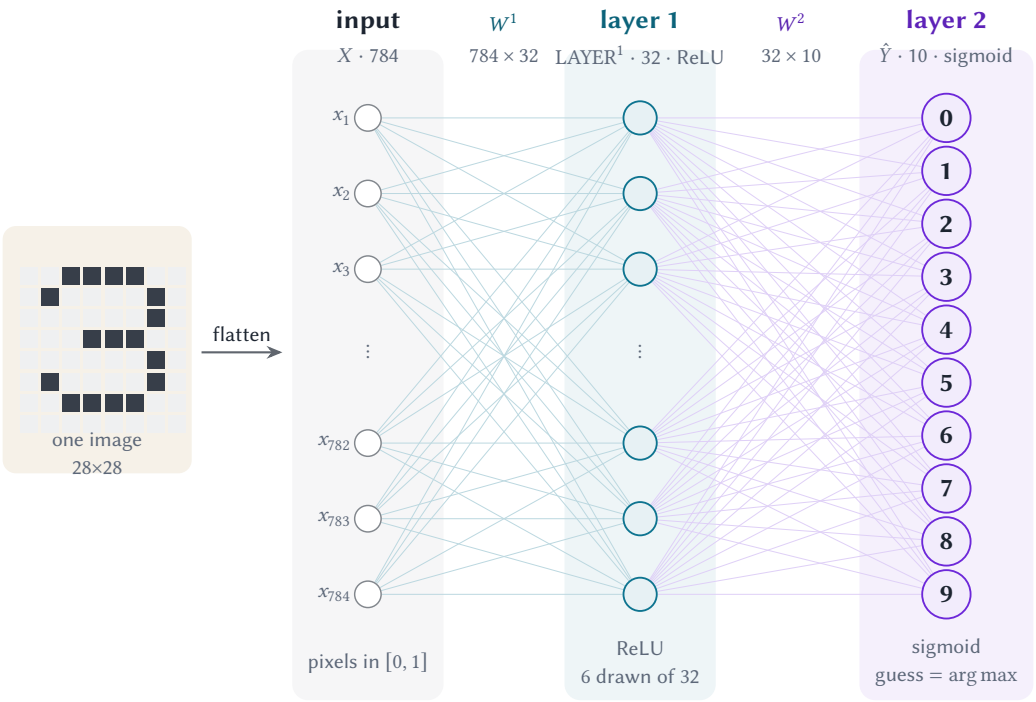


Figure 3: Structure of the network. The grid stands for one 28 × 28 digit, flattened into the 784-vector X. Six input circles and six hidden circles are drawn so the picture stays readable; the real hidden layer has 32 neurons, and every input connects to every one of them. All ten output scores are drawn. W^1 and W^2 are the only learned tables. There is no bias.

The pale lines are the idea of a full connection, not a plot of individual weights. Shapes that actually multiply:

object	meaning	shape
X	150 images, 784 pixels each	150 × 784
W^1	weights of every hidden neuron	784 × 32
Z^1	hidden weighted sums	150 × 32
LAYER^1	hidden activations	150 × 32
W^2	weights of every output neuron	32 × 10
Z^2	output weighted sums	150 × 10
$\hat{Y}$	10 scores per image	150 × 10
Y	true labels, one-hot	150 × 10

The notes label W^1 as 32 × 150 and W^2 as 10 × 150. Those sizes do not multiply with X (150 × 784). We keep the 32 and the 10 neurons, and we use 784 × 32 and 32 × 10 so the products are valid.

Forward pass

The schema, with σ meaning "an activation" in general:

Forward schema

$$X \to \otimes W^1 \to Z^1 \to \sigma \to \text{LAYER}^1 \to \otimes W^2 \to Z^2 \to \sigma \to \hat{Y} \;\gg\; \text{LOSS}(\hat{Y}, Y)$$

In this project the two activations are different functions.

$$\begin{aligned} Z^1 &= XW^1 \\ \text{LAYER}^1 &= \text{ReLU}(Z^1) \\ Z^2 &= \text{LAYER}^1\, W^2 \\ \hat{Y} &= \text{sigmoid}(Z^2) \end{aligned}$$

ReLU, on the hidden layer. $\text{ReLU}(z) = \max(0, z)$. Negative sums become 0; positive sums pass through. The slope is 1 when $z > 0$, and 0 otherwise.

Sigmoid, on the output layer.

$$\text{sigmoid}(z) = \frac{1}{1 + e^{-z}}.$$

Each of the 10 numbers is squashed into $(0, 1)$ and treated as a soft score for that digit. They are not forced to sum to 1 (that would be softmax, which is out of scope). The guess is arg max: the column with the largest score.

Loss

The true label is a one-hot row. Digit 3 is $(0, 0, 0, 1, 0, 0, 0, 0, 0, 0)$.

Mean squared error compares every score with every one-hot entry:

$$\text{LOSS} = (\hat{Y} - Y)^2,$$

summed over the batch in the code. If the network is sure about the right digit, $\hat{Y}$ is close to Y and the loss is small. If it is sure about the wrong digit, the loss is large.

We do not chase an accuracy percentage. The loss exists so the backward pass has a number to reduce.

Backward pass

Learning is backpropagation plus SGD. Measure the error at the output, send it backward through each layer, and nudge every weight a little so the next forward pass is less wrong.

The derivatives we need:

$$\begin{aligned} \text{sigmoid}'(z) &= \sigma(z)\,(1 - \sigma(z)), \\ \text{ReLU}'(z) &= \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{otherwise.} \end{cases} \end{aligned}$$

$\odot$ is element-wise multiply. The dot is matrix multiply. $(\cdot)^\top$ is transpose.

For one batch:

1. Error at the output (MSE composed with the sigmoid):

$$E = 2(\hat{Y} - Y) \odot \text{sigmoid}'(Z^2).$$

2. How the second-layer weights should change:

$$\frac{\partial \text{LOSS}}{\partial W^2} = (\text{LAYER}^1)^\top E.$$

   For each of the 32 hidden values: how much did it push each of the 10 output errors?

3. Send that error back through W^2 and through ReLU:

$$\frac{\partial \text{LOSS}}{\partial W^1} = X^\top \left(\left(E\,(W^2)^\top \right) \odot \text{ReLU}'(Z^1) \right).$$

   For each of the 784 pixels: how much did it push each of the 32 hidden neurons, given the error that arrived from the output?

4. SGD step, with learning rate $\eta = 1$:

$$W^1 \leftarrow W^1 - \eta \frac{\partial \text{LOSS}}{\partial W^1}, \qquad W^2 \leftarrow W^2 - \eta \frac{\partial \text{LOSS}}{\partial W^2}.$$

No momentum, no Adam, no regularization. A **batch** is 150 images. An **epoch** is one walk through the about 1,000 training images (about seven batches). Training runs for 50 epochs.

One practical note lives next to the update in the code. The formulas above differentiate a *summed* MSE. That sum, over a 150×10 batch, is far too large for $\eta = 1$ (the sigmoid saturates in a single step). The code divides the gradients by batch $\times$ 10, so the step is mean MSE while the printed loss stays the summed number.

Model definition

150 observations in a matrix X of shape 150×784: 150 rows, 784 columns.

Two weight matrices. The notes write W^1 as 32×150 and W^2 as 10×150. The locked shapes are $W^1 \in \mathbb{R}^{784\times32}$ and $W^2 \in \mathbb{R}^{32\times10}$.

Layers: a first layer of 32 neurons, a second layer of 10 neurons. The output has shape 150×10, one column per digit class.

Hidden activation: ReLU. Output activation: sigmoid.

Decisions

Language	Python, standard library only.
Purpose	Learn how a neural network works.
Stack	Pure Python, from scratch. No ML library and no NumPy.
Task	Handwritten digits on a MNIST subset.
Architecture	$784 \to 32 \to 10$.
Weights	W^1 is 784×32. W^2 is 32×10. No biases.
Predict	arg max of the 10 sigmoid scores, a digit 0–9.
Data	About 1,000 train images and 200 test images, in `mnist_subset.json.gz`.
Batch	150 images per step.
Epochs	50. Print the loss after each epoch, in training mode only.
Step size	$\eta = 1$, on mean MSE (see the note in the backward pass).
Weights file	`mnist_weights.json.gz`, so run mode can skip training.
Deliverable	One well-commented Python file: the theory, in code.

Run mode

Training mode is for watching the loss. Run mode is for seeing the network think.

- `python3 two_layer_mnist.py --train` trains and saves the weights.
- `python3 two_layer_mnist.py --run` loads the weights and draws. No epoch log.
- With no flag, the file trains, saves, then runs.

For one test image of each digit $0, \ldots, 9$, run mode:

1. Draws the 28×28 digit (sixel in Foot, text otherwise).
2. Draws LAYER[1]: the 32 ReLU values as a 4×8 heat grid of fixed-width cells, plus a one-line sparkline. Cells are scaled to that image's peak ReLU.

3. Draws LAYER^2: ten sigmoid bars, labeled 0 through 9. A star marks the guess. The true label and the guess use distinct highlights.
4. Prints one line: guess against truth.

   Color is soft ANSI when stdout is a terminal. NO_COLOR switches it off. No GUI toolkit.

Requirements

- From scratch, in pure Python: an explicit forward pass, loss, and backprop.
- Comments retell this theory next to the matching lines.
- No ML library, no NumPy, no other language.
- A MNIST subset file for training and for the guesses.
- Two layers, 32 then 10.
- One `.py` file, plus the modes above.

Success

- The Python file is simple enough to read as the forward, loss, backward, and update story.
- Training prints the loss each epoch and saves the weights.
- Run mode loads those weights and shows each sample digit with both layers.
- No accuracy number is required, and we do not tune toward one.

Constraints

Python, standard library only, on Vincent's machine. The path is this PRD, then the user stories, then `two_layer_mnist.py`.

Open questions

None.

Typeset from `docs/prd.md`. The network figure shows a sample of the connections; the implemented net is fully connected, 784 → 32 → 10, with ReLU then sigmoid and no biases.